



UNIVERSIDAD POLITÉCNICA SALESIANA
CARRERA DE INGENIERÍA AUTOMOTRIS

PROYECTO DE INTER CICLO

Docente: Ing. Carlos Carranco

INTEGRANTES

Osorio Fonseca Jhoel Mauricio

Narváez Peñafiel Rommel Daniel

Tipan Yaure Erick Paul

QUITO – ECUADOR

1. Tema:

Aplicación móvil que permita controlar y monitorear el sistema de iluminación de la tarjeta de prácticas mediante comunicación Bluetooth entre el teléfono y el microcontrolador Arduino

2. Objetivos

Objetivo General

Desarrollar una aplicación móvil que permita controlar y monitorear el sistema de iluminación de la tarjeta de prácticas mediante comunicación Bluetooth entre el teléfono y el microcontrolador Arduino, utilizando la tarjeta MCP2515 para la transmisión y recepción de mensajes sobre la red CAN BUS.

3. Diagramas

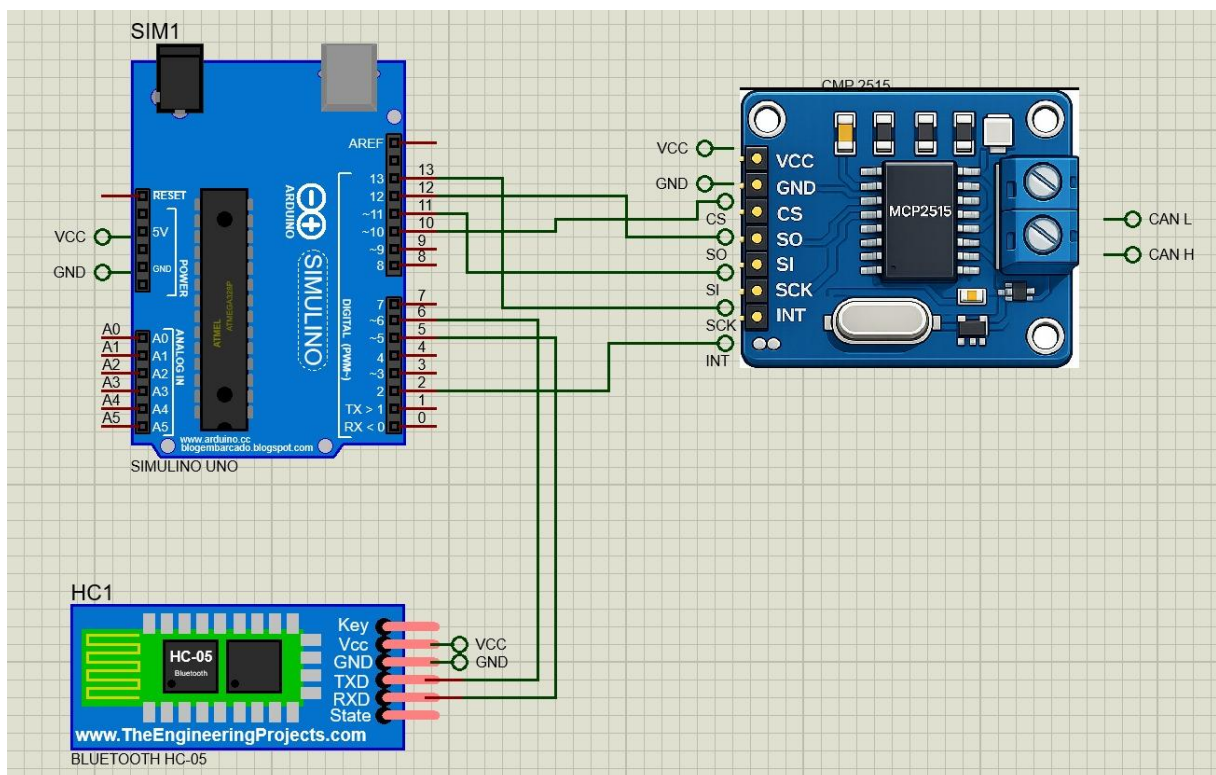


Ilustración 1 Diagrama

DIAGRAMA DE CONEXIÓN CAN BUS

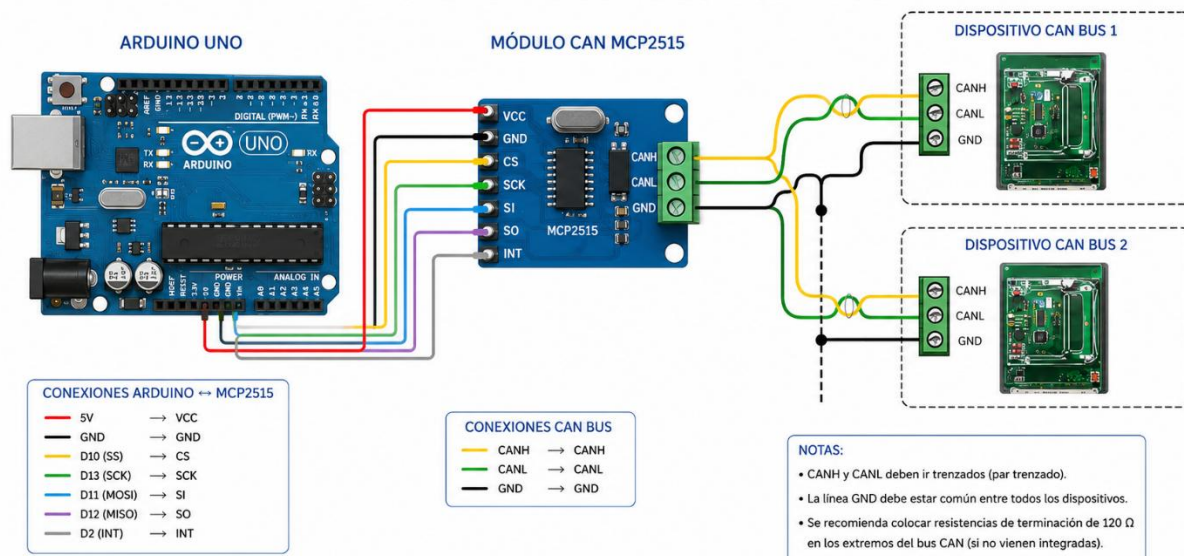


Ilustración 2 Diagrama Can Bus

FUNCIONAMIENTO DEL SISTEMA

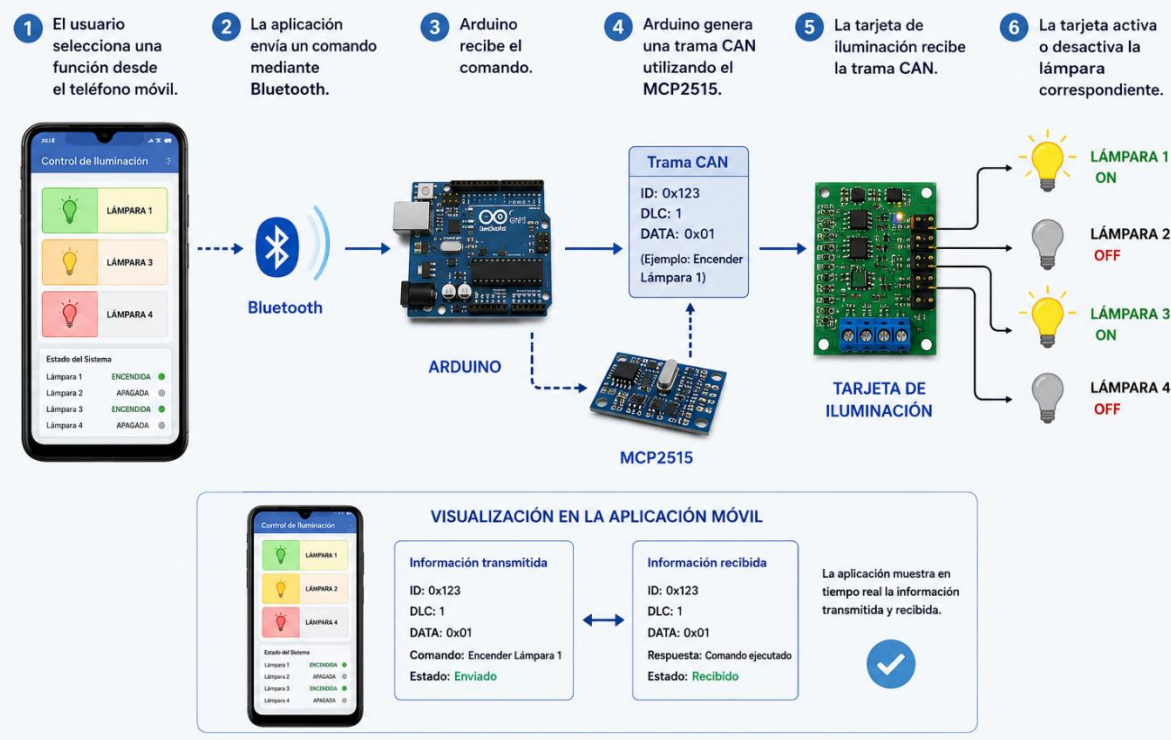


Ilustración 3 Funcionamiento del sistema

4. Código.

```
/*
=====
Control de comandos CAN vía Bluetooth (HC-05) - Arduino UNO + MCP2515
=====
- Bus CAN a 125 Kbps
- Cada función tiene una letra MAYÚSCULA para "ON/ABIERTO"
  y su equivalente en minúscula para "OFF/CERRADO"
*/

#include <SPI.h>
#include <mcp_can.h>
#include <SoftwareSerial.h>

// -----
// PINES
// -----
#define CAN_CS_PIN 10 // Pin CS del módulo MCP2515
#define BT_RX_PIN 3 // RX del Arduino <-- TXD del HC-05
#define BT_TX_PIN 4 // TX del Arduino --> RXD del HC-05 (con divisor de voltaje)

MCP_CAN CAN(CAN_CS_PIN);
SoftwareSerial BT(BT_RX_PIN, BT_TX_PIN);

// -----
// TABLA DE COMANDOS CAN
// -----
struct ComandoCAN {
    const char* nombre;
    unsigned long id;
    byte dlc;
    byte data[8];
};

ComandoCAN tabla[] = {
    {"PARQ_ON", 0x782, 3, {0x1F, 0x40, 0x48}},
    {"PARQ_OFF", 0x782, 3, {0x00, 0x40, 0x48}},

    {"FREN_ON", 0x783, 1, {0x08}},
    {"FREN_OFF", 0x783, 1, {0x00}},

    {"BAJAS_ON", 0x784, 1, {0x61}},
    {"BAJAS_OFF", 0x784, 1, {0x00}},

    {"ALTAS_ON", 0x784, 1, {0x63}},
    {"ALTAS_OFF", 0x784, 1, {0x61}},

    {"DIR_DER_ON", 0x782, 3, {0x3A, 0x40, 0x80}},
    {"DIR_DER_OFF", 0x782, 3, {0x00, 0x40, 0x80}},

    {"DIR_IZQ_ON", 0x782, 3, {0x25, 0x40, 0x80}},
    {"DIR_IZQ_OFF", 0x782, 3, {0x00, 0x40, 0x80}},

    {"PTA_DER_ON", 0x78D, 3, {0x01, 0x00, 0x00}},
```

SEDE QUITO – CAMPUS SUR

```
{"PTA_DER_OFF", 0x78D, 3, {0x00,0x00,0x00}},

{"PTA_IZQ_ON", 0x78C, 3, {0x01,0x00,0x00}},
{"PTA_IZQ_OFF", 0x78C, 3, {0x00,0x00,0x00}},

{"ABIERTO", 0x78A, 1, {0x0A}},
{"CERRADO", 0x78A, 1, {0x05}}
};

const int totalComandos = sizeof(tabla) / sizeof(tabla[0]);

// -----
// MAPA LETRA -> COMANDO
// -----
struct MapaLetra {
    char letra;
    const char* comando;
};

MapaLetra mapa[] = {
    {'P', "PARQ_ON"},    {'p', "PARQ_OFF"},    // Luces de parqueo
    {'F', "FREN_ON"},    {'f', "FREN_OFF"},    // Freno / stop
    {'B', "BAJAS_ON"},   {'b', "BAJAS_OFF"},   // Luces bajas
    {'H', "ALTAS_ON"},   {'h', "ALTAS_OFF"},   // Luces altas
    {'D', "DIR_DER_ON"}, {'d', "DIR_DER_OFF"}, // Direccional derecha
    {'I', "DIR_IZQ_ON"}, {'i', "DIR_IZQ_OFF"}, // Direccional izquierda
    {'R', "PTA_DER_ON"}, {'r', "PTA_DER_OFF"}, // Puerta derecha
    {'L', "PTA_IZQ_ON"}, {'l', "PTA_IZQ_OFF"}, // Puerta izquierda
    {'A', "ABIERTO"},    {'C', "CERRADO"}     // Seguro / chapa
};

const int totalMapa = sizeof(mapa) / sizeof(mapa[0]);

// -----
// SETUP
// -----
void setup() {
    Serial.begin(9600);
    BT.begin(9600); // Velocidad de fábrica típica del HC-05

    Serial.println("Iniciando MCP2515...");

    // OJO: si tu módulo tiene cristal de 16MHz, cambia MCP_8MHZ por MCP_16MHZ
    while (CAN_OK != CAN.begin(MCP_ANY, CAN_125KBPS, MCP_8MHZ)) {
        Serial.println("Error al iniciar CAN, reintentando...");
        delay(500);
    }

    CAN.setMode(MCP_NORMAL);
    Serial.println("CAN OK a 125 Kbps");
    Serial.println("Esperando comandos por Bluetooth...");
}

// -----
// LOOP
// -----
```

```
void loop() {
  if (BT.available()) {
    char c = BT.read();
    ejecutarLetra(c);
  }
}

void ejecutarLetra(char letra) {
  for (int i = 0; i < totalMapa; i++) {
    if (mapa[i].letra == letra) {
      enviarComandoPorNombre(mapa[i].comando);
      return;
    }
  }
  if (letra != '\n' && letra != '\r') {
    Serial.print("Letra no reconocida: ");
    Serial.println(letra);
  }
}

void enviarComandoPorNombre(const char* nombre) {
  for (int i = 0; i < totalComandos; i++) {
    if (strcmp(tabla[i].nombre, nombre) == 0) {
      byte resultado = CAN.sendMsgBuf(tabla[i].id, 0, tabla[i].dlc, tabla[i].data);
      if (resultado == CAN_OK) {
        Serial.print("Enviado -> ");
        Serial.println(nombre);
      } else {
        Serial.print("ERROR enviando -> ");
        Serial.println(nombre);
      }
    }
  }
  return;
}
}
```

5. Descripción del funcionamiento.

El programa tiene como objetivo controlar diferentes funciones de un sistema electrónico automotriz mediante el envío de mensajes CAN, utilizando un Arduino Uno, un módulo MCP2515 para la comunicación CAN y un módulo Bluetooth HC-05 para recibir comandos desde un teléfono móvil o cualquier dispositivo Bluetooth.

Al iniciar el sistema, el Arduino configura la comunicación serial con la computadora a **9600 baudios** y establece la comunicación con el módulo Bluetooth HC-05 a la misma velocidad. Posteriormente, inicializa el controlador **MCP2515**, configurándolo para trabajar en una red **CAN a 125 kbps**. Si el módulo CAN no logra inicializarse correctamente, el programa permanece intentando la conexión hasta que esta sea exitosa.

SEDE QUITO – CAMPUS SUR

Una vez configurado el hardware, el programa entra en un ciclo continuo ('loop') donde permanece esperando la llegada de un carácter enviado por Bluetooth. Cada carácter recibido representa una orden específica, por ejemplo encender luces, activar direccionales, abrir puertas o accionar el seguro de las puertas.

Cuando se recibe una letra, la función `**`ejecutarLetra()`**` busca esa letra dentro de una tabla de correspondencias denominada `**`mapa`**`, la cual relaciona cada carácter con un nombre de comando CAN. Las letras mayúsculas representan generalmente la activación de una función (ON o apertura), mientras que las minúsculas representan su desactivación (OFF o cierre).

- * **Identificador (ID) CAN.**
- * **Longitud del mensaje (DLC).**
- * **Bytes de datos que serán enviados.**

Con esta información, el programa utiliza la instrucción:
para transmitir el mensaje sobre la red CAN mediante el módulo MCP2515.

Después del envío, el programa verifica si la transmisión fue correcta. Si el mensaje fue enviado exitosamente, muestra en el Monitor Serial el nombre del comando transmitido; en caso contrario, informa que ocurrió un error durante la transmisión.

1. Inicializa el puerto serial, el módulo Bluetooth HC-05 y el controlador CAN MCP2515.
2. Espera continuamente la recepción de un carácter mediante Bluetooth.
3. Identifica el comando asociado a la letra recibida.
4. Busca el mensaje CAN correspondiente en la tabla de comandos.
5. Envía el mensaje a la red CAN utilizando el MCP2515.
6. Informa mediante el Monitor Serial si el envío fue exitoso o si ocurrió un error.

Este método basado en tablas permite que el código sea `**modular, escalable y fácil de mantener**`, ya que para agregar nuevas funciones únicamente es necesario incorporar un nuevo comando en la tabla CAN y asociarle una letra en el mapa de comandos, sin modificar la lógica principal del programa.

SEDE QUITO – CAMPUS SUR

6. Tabla de IDs utilizados.

Función	Comando	ID CAN (Hex)	DLC	Datos (HEX)
Luces de parqueo	PARQ_ON	0x782	3	1F 40 48
Luces de parqueo	PARQ_OFF	0x782	3	00 40 48
Luz de freno	FREN_ON	0x783	1	08
Luz de freno	FREN_OFF	0x783	1	00
Luces bajas	BAJAS_ON	0x784	1	61
Luces bajas	BAJAS_OFF	0x784	1	00
Luces altas	ALTAS_ON	0x784	1	63
Luces altas	ALTAS_OFF	0x784	1	61
Direccional derecho	DIR_DER_ON	0x782	3	3A 40 80
Direccional derecho	DIR_DER_OFF	0x782	3	00 40 80
Direccional izquierdo	DIR_IZQ_ON	0x782	3	25 40 80
Direccional izquierdo	DIR_IZQ_OFF	0x782	3	00 40 80
Puerta derecha	PTA_DER_ON	0x78D	3	01 00 00
Puerta derecha	PTA_DER_OFF	0x78D	3	00 00 00
Puerta izquierda	PTA_IZQ_ON	0x78C	3	01 00 00
Puerta izquierda	PTA_IZQ_OFF	0x78C	3	00 00 00
Seguro abierto	ABIERTO	0x78A	1	0A
Seguro cerrado	CERRADO	0x78A	1	05

Resumen de IDs utilizados

ID CAN	Función
0x782	Luces de parqueo y direccionales
0x783	Luz de freno (Stop)
0x784	Luces bajas y altas
0x78A	Seguro de puertas (Abrir/Cerrar)
0x78C	Puerta izquierda
0x78D	Puerta derecha

7. DLC de cada mensaje.

DLC	Cantidad de bytes	Mensajes
1	1 byte de datos	FREN_ON, FREN_OFF, BAJAS_ON, BAJAS_OFF, ALTAS_ON, ALTAS_OFF, ABIERTO y CERRADO

SEDE QUITO – CAMPUS SUR

3	3 bytes de datos	PARQ_ON, PARQ_OFF, DIR_DER_ON, DIR_DER_OFF, DIR_IZQ_ON, DIR_IZQ_OFF, PTA_DER_ON, PTA_DER_OFF, PTA_IZQ_ON y PTA_IZQ_OFF
---	------------------	--

8. Formato de los datos enviados.

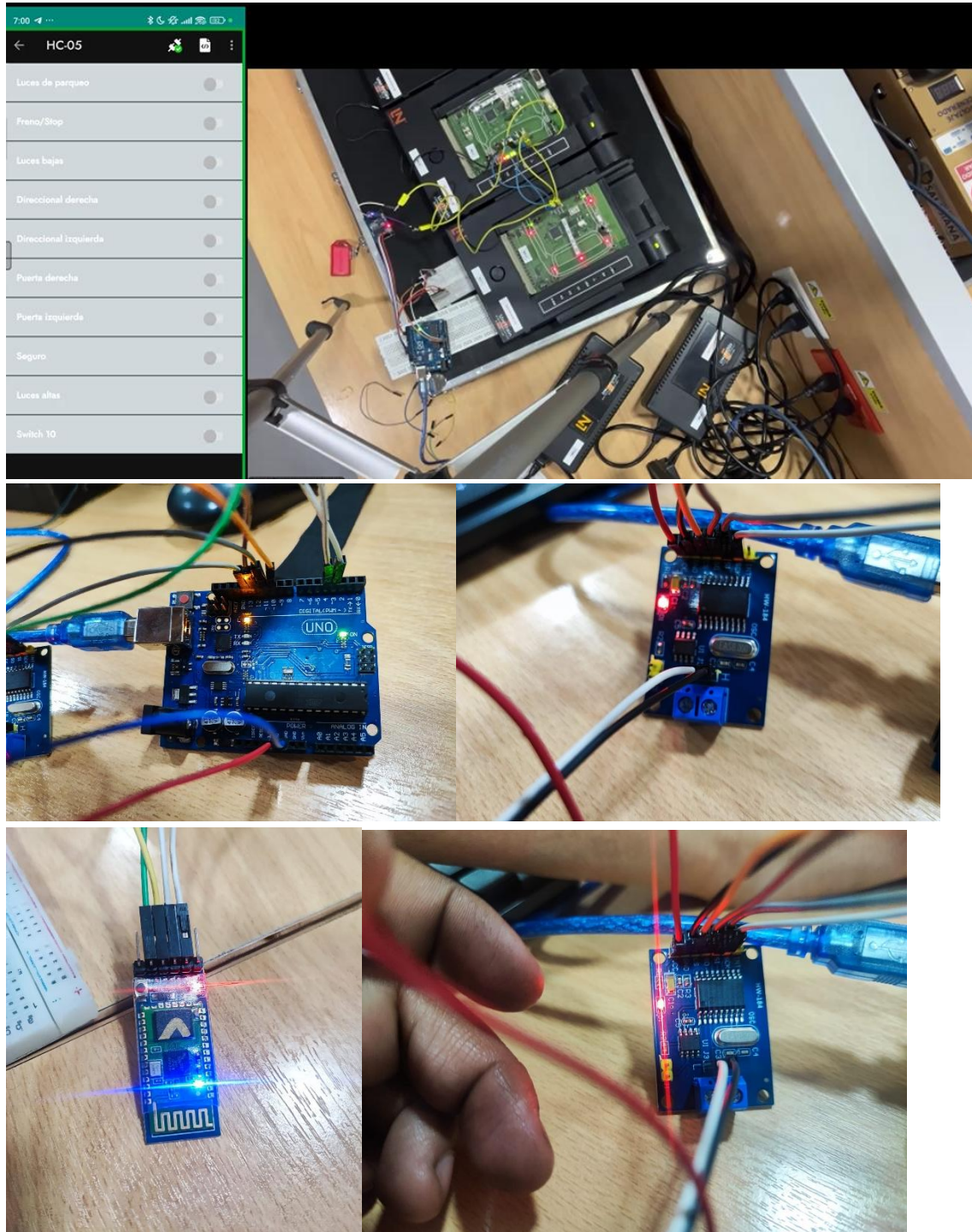
Cada mensaje transmitido en el proyecto utiliza el formato de una **trama CAN estándar (11 bits de identificación)** y está compuesto por los siguientes campos:

Campo	Descripción
ID CAN	Identificador del mensaje (11 bits). Determina la función que realizará el módulo receptor.
RTR	0 (Trama de datos). Indica que el mensaje contiene información y no es una solicitud remota.
DLC	Número de bytes de datos transmitidos (1 o 3 bytes en este proyecto).
DATA	Información enviada al nodo CAN en formato hexadecimal.

Formato de las tramas utilizadas

Comando	ID CAN	DLC	Datos (Hexadecimal)
PARQ_ON	0x782	3	1F 40 48
PARQ_OFF	0x782	3	00 40 48
FREN_ON	0x783	1	08
FREN OFF	0x783	1	00
BAJAS ON	0x784	1	61
BAJAS OFF	0x784	1	00
ALTAS ON	0x784	1	63
ALTAS OFF	0x784	1	61
DIR DER ON	0x782	3	3A 40 80
DIR DER OFF	0x782	3	00 40 80
DIR IZQ ON	0x782	3	25 40 80
DIR_IZQ_OFF	0x782	3	00 40 80
PTA_DER_ON	0x78D	3	01 00 00
PTA_DER_OFF	0x78D	3	00 00 00
PTA_IZQ_ON	0x78C	3	01 00 00
PTA_IZQ_OFF	0x78C	3	00 00 00
ABIERTO	0x78A	1	0A
CERRADO	0x78A	1	05

9. Evidencias



10. Conclusiones

- Se implementó exitosamente un sistema de comunicación CAN Bus utilizando un Arduino Uno, un módulo MCP2515 y un módulo Bluetooth HC-05, permitiendo controlar diferentes funciones del vehículo mediante el envío de tramas CAN.
- La utilización de una tabla de comandos y un mapa de letras simplificó la programación, haciendo que el código sea modular, organizado y fácilmente escalable para incorporar nuevas funciones o mensajes CAN.
- Se comprobó que cada acción del sistema se ejecuta mediante el envío de un identificador (ID CAN) específico, un DLC y un conjunto de datos en formato hexadecimal, respetando la estructura de una trama CAN estándar.
- La comunicación Bluetooth permitió establecer una interfaz inalámbrica sencilla entre el usuario y el sistema, facilitando el envío de comandos sin necesidad de conexiones físicas al Arduino.
- El proyecto permitió comprender el funcionamiento práctico del protocolo CAN Bus, desde la construcción y transmisión de mensajes hasta la activación de funciones como luces, direccionales, freno, puertas y seguros, demostrando su importancia en las redes electrónicas automotrices modernas.
- La implementación desarrollada constituye una base para futuras aplicaciones de diagnóstico, automatización y control electrónico en vehículos, pudiendo ampliarse con la incorporación de nuevos nodos CAN, sensores y sistemas de monitoreo en tiempo real.